Language Models Implement Simple Word2Vec-style Vector Arithmetic

Jack Merullo
Department of Computer Science
Brown University
jack_merullo@brown.edu

Carsten Eickhoff
School of Medicine
University of Tübingen
carsten.eickhoff@uni-tuebingen.de

Ellie Pavlick
Department of Computer Science
Brown University
ellie_pavlick@brown.edu

arXiv:2305.16130v3 [cs.CL] 3 Apr 2024

Abstract

A primary criticism towards language models (LMs) is their inscrutability. This paper presents evidence that, despite their size and complexity, LMs sometimes exploit a simple vector arithmetic style mechanism to solve some relational tasks using regularities encoded in the hidden space of the model (e.g., Poland:Warsaw::China:Beijing). We investigate a range of language model sizes (from 124M parameters to 176B parameters) in an in-context learning setting, and find that for a variety of tasks (involving capital cities, uppercasing, and past-tensing) a key part of the mechanism reduces to a simple additive update typically applied by the feedforward (FFN) networks. We further show that this mechanism is specific to tasks that require retrieval from pretraining memory, rather than retrieval from local context. Our results contribute to a growing body of work on the interpretability of LMs, and offer reason to be optimistic that, despite the massive and non-linear nature of the models, the strategies they ultimately use to solve tasks can sometimes reduce to familiar and even intuitive algorithms.[1]

1 Introduction

The growing capabilities of large language models (LLMs) have led to an equally growing interest in understanding how such models work under the hood. Such understanding is critical for ensuring that LLMs are reliable and trustworthy once deployed. Recent work in interpretability has contributed to this understanding by reverse-engineering the data structures and algorithms that are implicitly encoded in the model's weights, e.g., by identifying detailed circuits (Wang et al., 2022; Elhage et al., 2021; Olsson et al., 2022) or by identifying mechanisms for factual storage and retrieval which support intervention and editing (Geva et al., 2021a; Li et al., 2022; Meng et al., 2022a,c; Dai et al., 2022).

Here, we contribute to this growing body of work by analyzing how LLMs recall information during in-context learning. Modern LLMs are based on a complex transformer architecture (Vaswani et al., 2017) which produces contextualized word embeddings (Peters et al., 2018; Devlin et al., 2019) connected via multiple non-linearities. Despite this, we find that LLMs implement a basic vector-addition mechanism qualitatively similar to relational information encoded in their static word embeddings predecessors Mikolov et al. (2013). We also find that for non-injective relations that static embeddings typically fail to encode (Gladkova et al., 2016), LMs do not use the identified mechanism (Appendix G).

We study this phenomenon across nine tasks, but focus on three in the main paper: recalling capital cities, uppercasing tokens, and past-tensing verbs. Our key findings are:

- We find evidence of a **distinct processing signature** in the forward pass which characterizes argument-function processing (§3). That is, if models need to perform the `get_capital(`x`)` function, which takes an argument x and yields an answer y, they first surface the argument x in earlier layers which enables them to apply the function and yield y as the final output (Figure 2). This signature generalizes across models and tasks, but appears to become sharper as models increase in size.

- We take a closer look at GPT2-Medium, and find that the vector arithmetic mechanism is often implemented by mid-to-late layer feed-forward networks (FFNs) in a way that is **modular and supports intervention** (§4). E.g., an FFN outputs a content-independent update which produces *Warsaw* given *Poland* and can

[1]Code available at: https://github.com/jmerullo/lm_vector_arithmetic

be patched into an unrelated context to produce *Beijing* given *China*. We don't find this evidence of this mechanism being used for tasks in which word embedding vector arithmetic classically fails (Appendix G).

- We demonstrate that this mechanism is **specific to recalling information from pretraining memory** (§5). For settings in which the correct answer can be retrieved from the prompt, this mechanism does not appear to play any role, and FFNs can be ablated entirely with relatively minimal performance degradation. Thus, we present new evidence supporting the claim that FFNs and attention specialize for different roles, with FFNs supporting factual recall and attention copying and pasting from local context.

Taken together, our results offer new insights about one component of the complex algorithms that underlie in-context learning. The mechanism's simplicity raises the possibility that other apparently complicated behaviors may be supported by a sequence of simple operations under the hood. Moreover, our results suggest a distinct processing signature and hint at a method for intervention. These ideas could support future work on detecting and preventing unwanted behavior by LLMs at runtime.

2 Methods

In decoder-only transformer language models (Vaswani et al., 2017), a sentence is processed one word at a time, from left to right. In this paper, we focus on the transformations that the next-token prediction undergoes in order to predict the answer to some task. At each layer, an attention module and feed-forward network (FFN) module apply subsequent additive updates to this representation. Consider the FFN update at layer i, where x_i is the current next-token representation. The update applied by the FFN here is calculated as $\text{FFN}(\vec{x_i}) = \vec{o_i}, \;\; \vec{x_{i+1}} = \vec{x_i} + \vec{o_i}$ where $\vec{x_{i+1}}$ is the updated token for the next layer. Due to the *residual connection*, the output vector $\vec{o_i}$ is added to the input. $\vec{x}$ is updated this way by the attention and FFNs until the end of the model, where the token is decoded into the vocab space with the language modeling head E: $\text{softmax}(E\vec{x})$. From start to end, x is only updated by additive updates, forming a *residual stream* (Elhage et al., 2021). Thus, the token representation x_i represents all of the additions made into the residual stream up to layer i.

2.1 Early Decoding

A key insight from the residual stream perspective is that we can decode the next token prediction with the LM head before it reaches the final layer. This effectively allows for "print statements" throughout the model's processing. The intuition behind this technique is that LMs incrementally update the token representation $\vec{x}$ to build and refine an encoding of the vocabulary distribution. This technique was initially introduced in nostalgebraist (2020) as the logit lens, and Geva et al. (2022b) show that LMs do in fact refine the output distribution over the course of the model. Figure 1 illustrates the process we use to decode hidden states into the vocabulary space using the pre-trained language modeling head E. After decoding, we apply a softmax to get a probability distribution over all tokens. When we decode at some layer, we say that the most likely token in the resulting vocab distribution is currently being represented in the residual stream. We examine the evolution of these predictions over the course of the forward pass for several tasks.

2.2 Tasks

We apply early decoding to suite of in-context learning tasks to explore the transformations the next token prediction undergoes in order to predict the answer.

World Capitals The World Capitals task requires the model to retrieve the capital city for various states and countries in a few-shot setting. The dataset we use contains 248 countries and territories. A one-shot example is shown below:

"Q: What is the capital of France? A: Paris Q: What is the capital of Poland? A:___" Expected Answer: " Warsaw"

Reasoning about Colored Objects We focus on a subset of 200 of the reasoning about colored objects dataset prompts (i.e., the colored objects dataset) from BIG-Bench (Srivastava et al., 2022). A list of colored common objects is given to the model before being asked about one object's color. For the purposes of this paper, we focus only on one aspect of this task–the model's ability to output the final answer in the correct format.[2]

[2]The reason for this is that most of the results in this paper were originally observed as incidental findings while studying the Colored Objects task more generally. We thus zoom in on this one component for the purposes of the mechanism studied here, acknowledging that the full task involves many other steps that will no doubt involve other types of mechanisms.

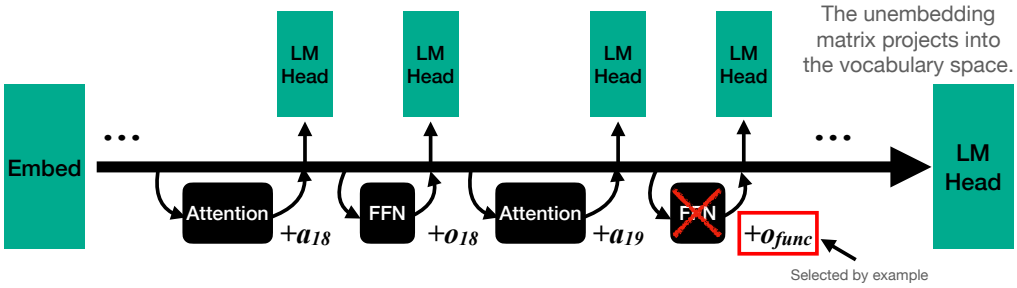

Figure 1: When decoding the next word, additive updates are made through the residual connections of each attention/FFN sub-layer. To decode the running prediction at every layer, the pre-trained language modeling head is applied at various points in each layer as in Geva et al. (2022a); nostalgebraist (2020). The $\vec{o}$ vector interventions we make (§4.1) are illustrated by patching one or more FFN outputs with one from another example

"Q: On the floor, I see a silver keychain, [...] and a blue cat toy. What color is the keychain?
A: Silver
Q: On the table, you see a brown sheet of paper, a red fidget spinner, a blue pair of sunglasses, a teal dog leash, and a gold cup. What color is the sheet of paper?
A:___" Expected answer: " Brown"

Past Tense Verb Mapping Lastly, we examine whether an LM can accurately predict the past tense form of a verb given a pattern of its present tense. The dataset used is the combination of the regular and irregular partitions of the past tense linguistic mapping task in BIG-Bench (Srivastava et al., 2022). After filtering verbs in which the present and past tense forms start with the same token, we have a total of 1,567 verbs. An example one-shot example is given below:

"Today I abandon. Yesterday I abandoned. Today I abolish. Yesterday I___" Expected answer: " abolished"

The above tasks could all be described as one-to-one (e.g., each country has one capital, each word only has one uppercase/past tense form). In Appendix G we explore six additional tasks, three of which are either many-to-many or many-to-one. We find that the observed mechanism only applies to one-to-one relations, indicating that the model learns some sensitivity to this type of relation in order for it to represent the structure required for the mechanism described here, similar to static embeddings (Gladkova et al., 2016)/

2.3 Models

We experiment on decoder-only transformer LMs across various sizes and pre-training corpora. When not specified, results in figures are from GPT2-medium. We also include results portraying

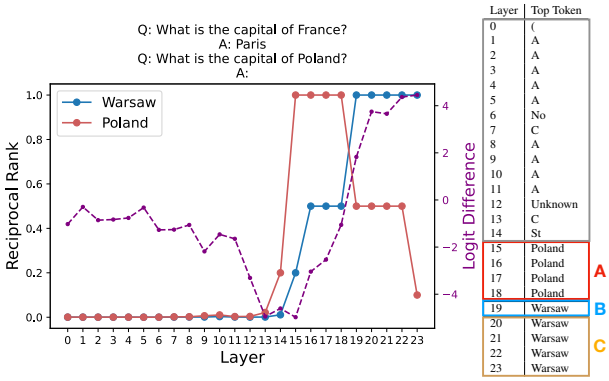

Figure 2: Decoding the next token prediction at each layer reveals distinct stages of processing. The red box (A) shows where the model prepares an argument for transformation, the blue box (B) shows the function application phase during which the argument is transformed (here with the `capital_of` function, and the yellow box (C) shows a saturation event, in which the model has found the answer, and stops updating the top prediction. The dashed line shows the logit difference between argument and answer at each layer.

the stages of processing signatures in the residual streams of the small, large, and extra large variants (Radford et al.), the 6B parameter GPT-J model (Wang and Komatsuzaki, 2021), and the 176B BLOOM model (Scao et al., 2022), either in the main paper or in the Appendix.

3 Stages of Processing in Predicting the Next Token

First, we use the early decoding method in order to investigate how the processing proceeds over the course of a forward pass to the model. Each task requires the model to infer some relation to recall some fact, e.g., retrieving the capital of Poland. In these experiments, we see several discrete stages of processing that the next token undergoes before